Lottery Search System

Design Proposal

Design-only proposal • No code implementation

1. Overview

The system searches a large collection of six-digit lottery tickets using patterns that contain digits and the wildcard character (*). The design should keep searches fast while ensuring a ticket is not assigned to more than one user at the same time.

I would use PostgreSQL as the source of truth for ticket data and reservations. For faster pattern matching at the target scale, I would use a positional bitmap index to narrow down candidate tickets, then let PostgreSQL make the final allocation decision. This keeps search and allocation separate: the index helps find candidates, while the database guarantees correctness.

2. Proposed Architecture

The main components are:

- API service: validates the six-character pattern and handles user requests.
- Search index: maintains a bitmap for each digit at each position, such as position 6 = digit 3.
- PostgreSQL: stores ticket records, availability, reservation owner, and reservation expiry. It is the authoritative source for allocation.
- Reservation flow: checks candidate IDs from the search index, then atomically reserves an available ticket in PostgreSQL.

The bitmap index can initially be built when the service starts and refreshed as tickets change. If the dataset or traffic grows, it can be maintained by a background worker or rebuilt from PostgreSQL. A stale index may return extra candidates, so every candidate must be checked against the current database state before it is allocated.

3. Data Model and Storage

Each physical ticket should have its own unique ticket_id, even when two tickets contain the same six-digit number. The number should be stored as a string or as six digit fields so leading zeros are preserved.

Field	Purpose
ticket_id	Unique ID for each physical ticket
number	Six-digit number, stored without losing leading zeros
status	AVAILABLE, RESERVED, or SOLD/ASSIGNED, depending on business rules
reserved_by	User who currently holds the reservation; nullable
reserved_until	Expiry time for temporary reservations; nullable

PostgreSQL is a practical choice because it supports indexes, transactions, row-level locks, constraints, and operational tools in one system. It is a good fit when correctness matters more than using a separate database for every task. The exact schema can use generated digit columns or a separate digit representation to support indexing.

4. Pattern Search and Indexing

The pattern must contain exactly six characters. Each character must be a digit or *. For example, ****23 fixes positions 5 and 6; 1****5 fixes positions 1 and 6; and 123*** fixes positions 1, 2, and 3.

For search, assign each ticket a numeric ticket_id and create six positional bitmap groups. Each group contains ten bitmaps, one for each digit 0–9. For example, the bitmap for position 6 and digit 3 contains the IDs of all tickets whose last digit is 3.

To search a pattern, start with the bitmap of all tickets and intersect it with the bitmap for each fixed digit. Wildcards are ignored. The remaining IDs are candidates. For example, ****23 is the intersection of position 5 = 2 and position 6 = 3. A compressed bitmap implementation such as RoaringBitmap can reduce memory and make intersections fast.

The bitmap index is an optimization, not the source of truth. PostgreSQL still verifies that a candidate matches the pattern and is AVAILABLE. For a simpler first version, PostgreSQL can also use indexes on the six digit columns, with additional composite indexes for common prefix and suffix searches. The bitmap approach is more flexible for arbitrary combinations of fixed positions, but adds index maintenance complexity.

5. Safe Allocation Under Concurrency

Searching and reserving must not be separate unprotected operations. Two requests can find the same available ticket at nearly the same time. The reservation step therefore has to be atomic in PostgreSQL.

Suggested flow:

1. Use the bitmap index to produce candidate ticket IDs that match the pattern.
2. Start a database transaction and select an AVAILABLE candidate that still matches the pattern, using a row lock such as SELECT ... FOR UPDATE SKIP LOCKED.
3. Update that ticket to RESERVED, set reserved_by and reserved_until, and commit the transaction.
4. Return the ticket only after the transaction commits. If another request already reserved the candidate, skip it and try another candidate.

The database row lock and conditional status change prevent two concurrent requests from successfully reserving the same ticket. This should be enforced in the database transaction, not only in application memory. The rule should apply across all patterns, because different patterns can also match the same ticket.

Temporary reservations should expire after a defined period. A background job can release expired reservations back to AVAILABLE, but it must update rows conditionally so it does not release a ticket that has already been confirmed or sold. If clients retry requests, an idempotency key can prevent a retry from accidentally creating a second reservation.

6. Performance and Trade-offs

Area	Expected behavior / trade-off
Search	Bitmap intersections work on compact sets of ticket IDs rather than scanning all 10 million records for every request.
Memory	Six positions × ten digits gives 60 bitmaps. With 10 million IDs, the raw bitmap size is manageable on a suitable server; compression can reduce it further depending on ID distribution.

Area	Expected behavior / trade-off
Database load	Only candidate IDs need final availability checks and reservation writes. Large result sets should be paginated or limited.
Concurrency	PostgreSQL row locks and transactions provide a single authoritative allocation decision.
Complexity	A bitmap index is faster for flexible wildcard combinations but needs refresh/rebuild logic. A database-only design is simpler to operate but may be slower for frequent, varied patterns.

For 10 million tickets, this design should be practical on a properly sized server, but real performance must be confirmed with representative data and concurrent load tests. Important metrics include search latency (p95/p99), candidate count, allocation conflicts, database query time, and index refresh time.

7. Assumptions

- A ticket is identified by ticket_id, not only by its six-digit number, because multiple physical tickets may share the same number.
- A ticket can have at most one active reservation at a time.
- The exact reservation lifetime and what happens after payment or timeout are business rules to be confirmed.
- PostgreSQL is the source of truth; the search index may be rebuilt from it if needed.

In summary, I would use PostgreSQL for reliable storage and atomic reservations, with a positional bitmap index to make wildcard searches efficient. This balances search speed with the correctness needed when multiple users request matching tickets concurrently.